Convolutional networks

Flowers102

LECTURE 12 | GÉRON CH. 12

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Where we are

Lecture 11 trained a fully connected network with no labels, on 28×28 greyscale images (Fashion-MNIST) flattened to 784 numbers:

- An encoder squeezed each image to a code of 32 numbers and a decoder rebuilt it; the bottleneck stopped the network from copying
- A linear autoencoder reaches the principal subspace of Lecture 6; two ReLU layers went a sixth below that floor
- The code retrieved and clustered as well as the 784 pixels, and no better

WHAT BREAKS WHEN THE IMAGE GETS BIGGER

Flattening a 224×224 colour image gives 150,528 inputs. A single dense layer of 1,000 units on that is **150 million parameters** — for one layer, on one image size. Every one of them has to be learned from data, and none of them knows that two adjacent pixels are related.

Today's method fixes both problems at once, and the derivation says exactly how much it saves.

Today

1. The task, and why a flattened image is the wrong representation
2. **The mathematics** — what weight sharing saves, equivariance, and how pooling buys invariance
3. **The method** — convolution, pooling, an architecture, and training it
4. What it learned, what it scores, and what goes wrong quietly
5. **Design exercise** — a recycling-robotics company's camera sorter on its customers' conveyors, in nine steps, with the pitfalls and how to address them

Open Lecture 12 in Colab

The stakeholder

A horticultural wholesaler receives mixed pallets of cut stems and has to sort them by species before they reach the auction floor.

- One overhead camera per lane, one photograph per stem
- A human inspector currently does this and is the bottleneck
- They want the machine to name the species from the photograph

Same shape of problem as the garment sorter in Lecture 7. Everything that makes it harder is in the pixels.

What they actually asked for

> **IN THEIR WORDS**
>
> “Tell us which of our species it is. If you cannot tell, say so and we will look at it ourselves — but do not tell us the wrong one.”

Two requirements hiding in one sentence: a classifier over a fixed list of species, and a usable notion of *confidence*.

We build the first today. The second waits until the numbers are good enough to be worth calibrating.

> **FAILURE CONDITION — A CLASSIFIER WITH NO WAY TO ABSTAIN**
>
> The stakeholder asked to be told when the model cannot tell. A softmax always returns a class, and its largest probability is not a confidence you have calibrated. Deliver it as one and you have answered a different requirement while appearing to meet this one.

The dataset

`torchvision.datasets.Flowers102` — 8,189 photographs of 102 flowering species common in the United Kingdom.

```
from torchvision.datasets import Flowers102

train = Flowers102("data", split="train", download=True)
val   = Flowers102("data", split="val",   download=True)
test  = Flowers102("data", split="test",  download=True)
```

About 345 MB. It is one of the datasets the textbook names for this chapter, and the split used here is the one shipped with it — so every result in the literature is on the same 6,149 test images.

What the camera sees

12 of the 102 species, one image each, resized to 128 × 128



Different scales, different backgrounds, different lighting — and several of these species are hard to tell apart by eye.

What is hard here, precisely

- **Position** — the flower is not centred, and not at a fixed scale
- **Background** — leaves, soil, other flowers, a gardener's hand
- **Between-class similarity** — several of the 102 species are separated by petal count and little else
- **Within-class variation** — the same species at two stages of opening

Only the first of those has a structural answer. It is the answer this chapter is about.

The split we are given

Split	Images	Per class	Used for
train	1,020	10, exactly	fitting
val	1,020	10, exactly	choosing
✗ test	✗ 6,149	20–238	reporting, once

✗ There are six times as many test images as training images.

Why the training split is so small

Because somebody had to look at every photograph and name the species.

> **THE STANDING CONSTRAINT OF APPLIED VISION**
>
> Images are cheap and labels are not. Every application in this part of the course has more pixels than it can use and fewer labels than it wants.

Design for that. A method that needs thousands of labelled examples per class is not a method you can deploy here.

The obvious thing to try first

You have a multilayer perceptron that works. Resize every photograph to 128 × 128, flatten it, and feed it in.

	Count
Inputs, 128 × 128 × 3	49,152
Outputs, 32 values at every pixel position	524,288
✗ **Weights in one fully connected layer between them**	✗ **25,769,803,776**

✗ **That is 96 GB of float32 for one layer. The full argument is this lecture's mathematics block; here it is just a reason to do something else.**

What weight sharing saves

Input $3 \times H \times W$, output $32 \times H \times W$. A dense layer connects every output to every input: $n_{\text{in}} n_{\text{out}} = 96\,H^2 W^2$ weights. A convolution connects each output to a $k \times k$ window of every channel, **with the same weights at every position**: $C_{\text{out}} C_{\text{in}} k^2 = 32 \times 3 \times 7 \times 7$, which is 4,704 weights.

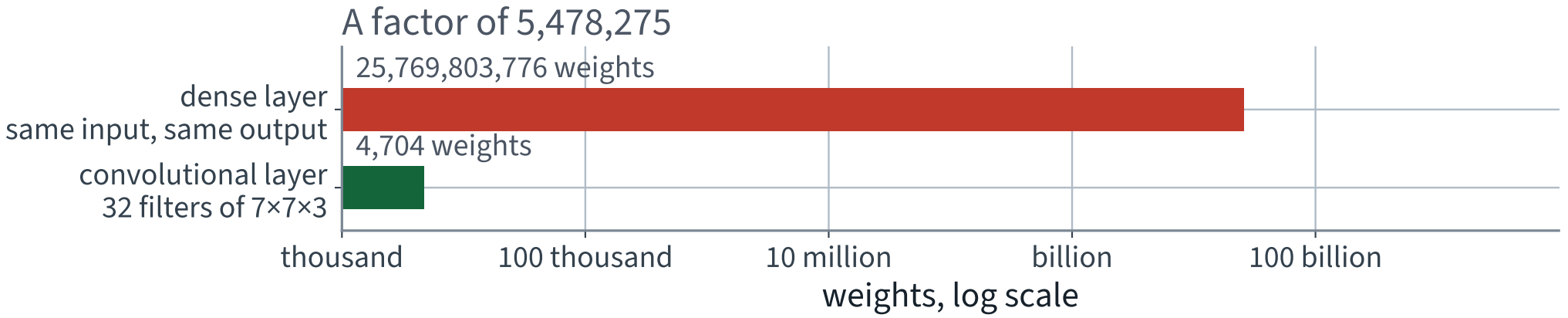
$$\frac{n_{\text{in}} n_{\text{out}}}{C_{\text{out}} C_{\text{in}} k^2} = \frac{H^2 W^2}{k^2}$$

5,478,275

times fewer weights at $H = W = 128$, for the same output shape.

With the 3 × 3 filters used everywhere else in the network the factor is larger again, by $(7/3)^2$: the dense count never contained k.

The same output shape, two ways



A logarithmic axis, and the two bars still barely fit on the same figure.

Equivariance, precisely

Sharing the weights assumes that a pattern means the same thing wherever it appears. Let $T_{\mathbf{v}}$ be the operator that shifts an image by the vector $\mathbf{v}$:

$$(T_{\mathbf{v}} x)(\mathbf{p}) \;=\; x(\mathbf{p} - \mathbf{v})$$

A map f is **equivariant** to translation when

$$f(T_{\mathbf{v}}\, x) \;=\; T_{\mathbf{v}}\, f(x) \qquad \text{for every } \mathbf{v}$$

Shift then compute, or compute then shift — the same answer. A convolution has this property because its weights do not depend on the position; it holds exactly away from the border, where zero padding invents input that was not there.

How pooling manufactures invariance

A map g is **invariant** to translation when

$$g(T_{\mathbf{v}}\, x) \;=\; g(x) \qquad \text{for every } \mathbf{v}$$

The output does not move at all, so it no longer says *where*. Classification wants that; equivariance alone does not give it.

A 2×2 max pool is invariant to any shift that keeps the maximum inside its own window — so half the one-pixel shifts, in each direction.

- Stack four pooling stages and the tolerated shift grows to ± 8 pixels in the last feature map
- A global pool over the whole map is invariant to *every* shift — there is no position left to be sensitive to

The invariance is approximate and it is built, one factor of two at a time. Nothing in the architecture declares it.

The idea, in one sentence

A feature worth detecting in the top left corner is worth detecting in the bottom right, so **use the same weights everywhere**.

- That is the whole of it. Everything else is bookkeeping
- It buys two things at once: far fewer parameters, and a network that does not have to relearn a petal edge at every position

Both halves were stated in the mathematics block. Here we use them.

A convolution, as arithmetic

For an input x with C channels and a kernel w of size $k \times k$, the output at position (i, j) of filter f is

$$z_{ij}^{(f)} = b^{(f)} + \sum_{c=1}^{C} \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} w_{c,u,v}^{(f)} \, x_{c,\, i+u,\, j+v}$$

A dot product between the kernel and a window of the input, repeated at every position. The sum over c is why a filter reads *all* the input channels at once.

`nn.Conv2d`, and the defaults you did not ask for

```
nn.Conv2d(
    in_channels=3,        # must match the previous layer's output
    out_channels=32,      # how many filters -> how many output maps
    kernel_size=7,
    stride=1,             # the default
    padding=3,            # NOT the default; the default is 0
    bias=False,           # a batch-norm follows, and it has its own shift
)
```

The default you did not ask for: **padding defaults to zero**, which silently shrinks every feature map by $k - 1$ and will eventually give you a shape error twelve layers later.

Pooling

After a couple of convolutions, halve the height and width by keeping only the largest value in each 2×2 block.

- Three quarters of the activations disappear, and with them three quarters of the memory and the arithmetic in every later layer
- It has **no parameters** — nothing about it is learned
- Each later unit therefore sees a larger piece of the original image

There is a fourth consequence — invariance — and it is the one the mathematics block ended on.

Max pooling, on numbers

4 × 4 feature map

5	3	8	1
2	9	4	0
1	6	7	3
4	2	5	8

after 2 × 2 max pool

9	8
6	8

Sixteen numbers in, four out, and the four are the largest of each block.

Why every convolution is followed by a batch norm

Because ten layers of untreated convolutions on 1,020 images do not train. Batch normalisation — Lecture 8's layer that uses the batch's statistics in training — rescales each channel to zero mean and unit variance over the batch, then applies a learned scale and shift, so the signal keeps its size through every layer.

- Ten layers of untreated convolutions on 1,020 images do not train: the loss sits at $\log 102 \approx 4.62$ and the network predicts one class
- `BatchNorm2d` normalises across the batch *and* both spatial axes, so it holds one mean and one variance per channel

```
nn.Conv2d(c_in, c_out, 3, padding=1, bias=False)
nn.BatchNorm2d(c_out)     # 2 * c_out parameters, not 2 * c_out * H * W
nn.ReLU()
```

The architecture

Stage	Layers	Output
1	conv 7×7 ×32, conv 3×3 ×32, pool	32 × 64 × 64
2	conv ×64, conv ×64, pool	64 × 32 × 32
3	conv ×128, conv ×128, pool	128 × 16 × 16
4	conv ×256, pool	256 × 8 × 8
✓ head	**✓ flatten, dense 256, dropout, dense 102**	**✓ 102**

Channels double as the map halves — the textbook's own rule of thumb: what pooling takes away in resolution, the extra filters give back in channels.

The architecture, in code

```
def conv_block(c_in, c_out, k=3):
    return [nn.Conv2d(c_in, c_out, k, padding=k // 2, bias=False),
            nn.BatchNorm2d(c_out), nn.ReLU()]

net = nn.Sequential(
    *conv_block(3, 32, k=7), *conv_block(32, 32), nn.MaxPool2d(2),
    *conv_block(32, 64),     *conv_block(64, 64), nn.MaxPool2d(2),
    *conv_block(64, 128),  *conv_block(128, 128), nn.MaxPool2d(2),
    *conv_block(128, 256),                        nn.MaxPool2d(2),
    nn.Flatten(),
    nn.Linear(256 * 8 * 8, 256), nn.ReLU(),
    nn.Dropout(0.5),
    nn.Linear(256, 102),
)
```

Thirty modules. Nothing in it that was not on a slide in this lecture or the previous one.

Check the shapes before you check anything else

```
x = torch.zeros(2, 3, 128, 128)
for m in net:
    x = m(x)
    print(f"{type(m).__name__:12s} {tuple(x.shape)}")

assert x.shape == (2, 102), f"the head is wrong: {x.shape}"
```

What is the shape here? Asking costs four lines. A shape error inside a training loop surfaces as a size-mismatch traceback two hundred lines deep; here it surfaces as a printed line.

Count the parameters, and look at where they are

Part	Parameters	Share
Every convolution and batch-norm layer together	586,720	12.2%
✗ One dense layer, 16,384 → 256	**✗ 4,194,560**	**✗ 87.3%**
Output layer, 256 → 102	26,214	0.5%
✓ Total	**✓ 4,807,494**	100%

✗ Nine parameters in ten are in the layer that is not convolutional.

The training loop — unchanged

```
opt   = torch.optim.Adam(net.parameters(), lr=3e-4)
lossf = nn.CrossEntropyLoss()

for epoch in range(30):
    net.train()
    for xb, yb in train_loader:
        opt.zero_grad()
        loss = lossf(net(xb), yb)
        loss.backward()
        opt.step()
```

Exactly the five lines from Lecture 8. Convolution changed the model; it did not change the loop, because autograd does not care what the modules are.

One hyperparameter that is not a free choice

Learning rate	Seed 42	Seed 43	Mean
✗ 10^{-3}, Adam's default	0.98%	1.76%	✗ 1.37%
✓ 3×10^{-4}	4.22%	6.76%	**5.49%** ✓

Validation accuracy after 12 epochs, on this architecture and this data. The default does not diverge — it plateaus at or near chance, which on a split with ten images of every species is 0.98%, and that is the failure mode that does not announce itself.

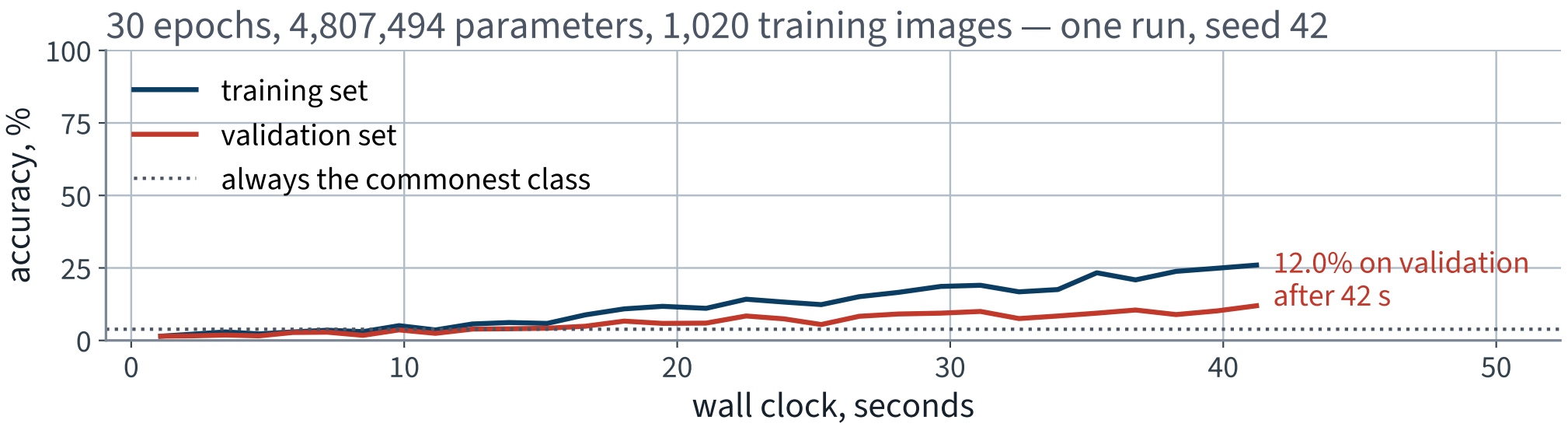
4.12 points, from one keyword argument nobody wrote down — about one and a half times the 2.55 points between the two seeds of the good rate.

Run it

Setting	Value
Epochs	30
Batch size	32
Steps per epoch	32
Total optimiser steps	960

A validation pass follows every epoch.

The cost, on the x-axis



The x-axis is seconds, not epochs. That is deliberate: from the next lecture on, time is one of the things being compared.

The test set. Once.

```
net.eval()
with torch.no_grad():
    correct = sum((net(xb).argmax(1) == yb).sum().item()
                  for xb, yb in test_loader)
print(f"test accuracy {correct / len(test_ds):.4f}")
```

8.13%

One run, seed 42. Two seeds of a shorter run of the same network land 2.55 points apart, so one run of a network this size on 1,020 images is an anecdote.

Why one run is not a result

Seed	Validation accuracy, 12 epochs
42	4.22%
43	6.76%

✗ 2.55 points between two runs — 26 of the 1,020 validation images — and nothing changed but the seed.

Any comparison you make from here that is smaller than that is not a comparison. Two seeds only show that the spread exists; a mean you would report needs more runs than a free CPU affords.

One seed reported as a result

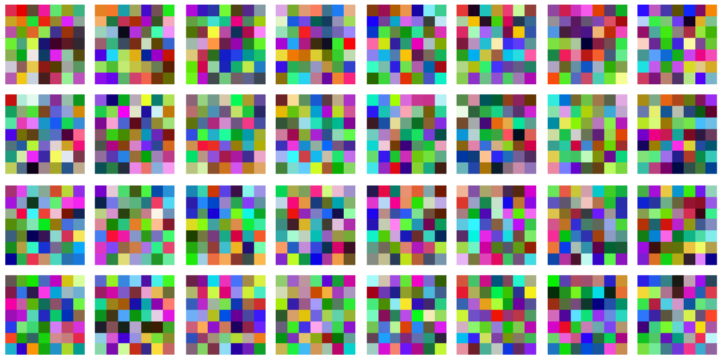
FAILURE CONDITION

2.55 points separate two runs with nothing changed but the seed. A single run is one draw from that spread, and a comparison of two single runs is two draws — which is why what gets reported is the spread as well as the mean.

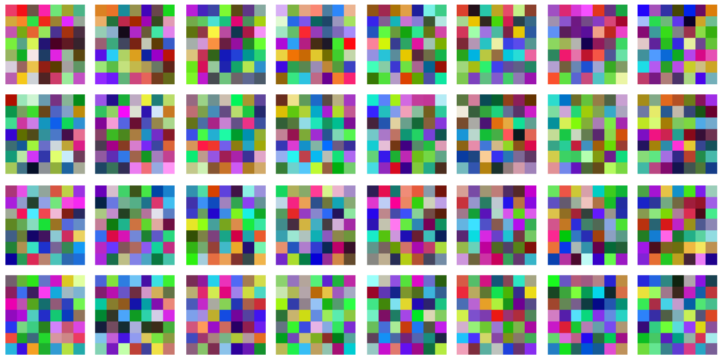
Thirty-two filters, before and after

All 32 filters of the first layer, 7 × 7 × 3, each rescaled to its own range

at initialisation



after 30 epochs



Look at the two blocks for ten seconds, and say what changed.

It moved a little. It did not become anything

```
import torch.nn.functional as F

# init_filters: a copy of net[0].weight, taken before the first step
before, after = init_filters.flatten(), net[0].weight.flatten()
print(f"cosine similarity {F.cosine_similarity(before, after, dim=0):.4f}")
print(f"mean |change| / mean |weight| "
      f"{(after - before).abs().mean() / before.abs().mean():.1%}")
```

	Value
✓ cosine similarity, all 4,704 weights before against after	**0.9880 ✓**
mean absolute change, as a share of the mean weight	14.1%
weights that changed sign	160

✗ The weights moved. No structure appeared — the right-hand block is still noise, and thirty epochs of training bought it.

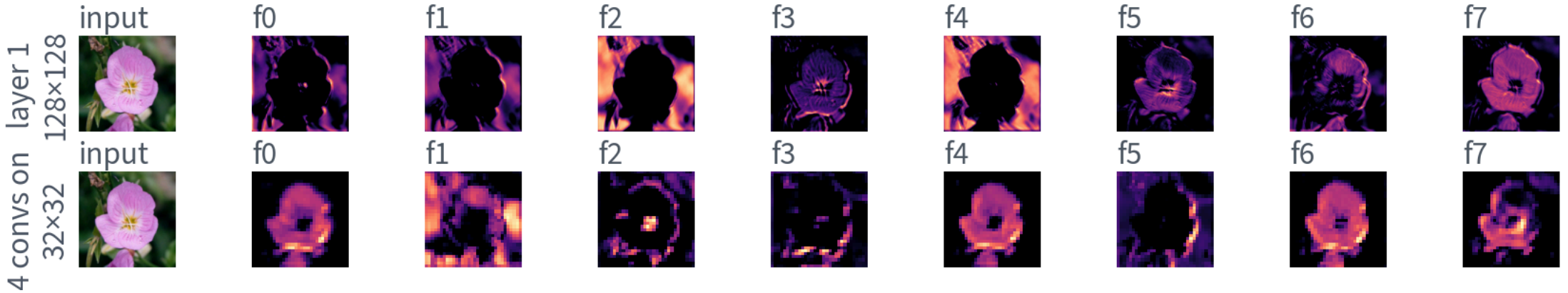
Two reasons, and both are worth knowing

1. **The gradient that reaches it is small.** Ten layers of chain rule separate the loss from these weights, and 1,020 images give 960 optimiser steps in total
2. **Its scale does not matter.** A batch norm follows immediately, and normalising the output makes the overall size of these weights irrelevant to the function — so there is no gradient pressure on it at all

The textbook picture of a first layer — colour blobs, oriented edges — is real, and you will see it next lecture. It comes from a corpus a thousand times this size.

What a filter does to one photograph

Feature maps: the first layer keeps the picture, the fourth keeps a summary



Early maps still look like the photograph. Four layers down they do not, and that is the point.

Reading a feature map

- Bright means *this filter fired strongly here* — nothing more mystical than that
- The spatial layout survives: a bright patch is where in the image the pattern was found
- After four pooling stages the map is 8×8, so one cell answers for a 256-pixel square of the original

That last line is a loss of information we chose. It is the right choice for naming a species, and the wrong one for any task that must say *where*.

The scoreboard

	Test accuracy
Uniform guess	0.98%
Commonest species	3.87%
✓ Yours, today	**8.13% ✓**
Operator’s requirement	90%

Above both baselines — 2.1 times the commonest species — and far below the requirement. Both distances matter.

What this number is and is not

- **Is**: one run, seed 42, of one architecture on one split, measured on 6,149 held-out images
- **Is not**: a claim about convolutional networks. It is a claim about this one, trained on 1,020 images
- Two seeds of a shorter run differed by 2.55 points. Any comparison smaller than that is not a comparison

Quote the spread whenever you quote the number. It is the difference between a measurement and an anecdote.

Five ways this build goes wrong quietly

Mistake	Symptom
`padding=0` left at the default	maps shrink; a shape error many layers later
Forgetting `net.eval()`	the score wobbles between calls — Lecture 8
Normalising with statistics of all splits	2.16 points, inside the seed noise
`lr` left at Adam's default	4.12 points, and no error
Pooling before any convolution	throws away the pixels before anything reads them

The notebook for this lecture

Open Lecture 12 in Colab

- **Runs on free CPU.** It trains 30 epochs, and every accuracy on these slides is one it printed
- The dense-against-convolutional parameter count, computed rather than quoted — the arithmetic you can do on paper first
- The network built and shape-checked layer by layer, the learning-rate and seed comparisons, one test score, and the first layer's filters and feature maps

> **WHAT TO CHANGE**
>
> Set `padding=0` on every convolution and re-run the shape check. Every map shrinks and the first linear layer no longer matches.

Design exercise · in the manner of an ML system-design interview

The task: a recycling-robotics company's camera sorter

> **THE BRIEF**
>
> A recycling-robotics company sells a sorting unit for its customers' conveyors: a camera photographs every object, an air jet blows it into the bin for its material. It names the **material** of each object — paper, cardboard, two plastics, glass, aluminium, residual — in time for the jet. Users: the plant operators. The business warrants the purity of the bales its customers sell, and pays for returns.

- **Given:** a few thousand photographs from installed units, one object each, labelled by the customers' operators
- **Scale:** several objects a second per belt; the jet fires a fraction of a second later
- **Consumers of the output:** the unit's diverter, the customer's purity report per bale, the support team

✓ Nine steps follow, each with the pitfall that catches a first attempt and what to do about it. The model at the centre is today's: a convolutional network trained from scratch.

Step 1 — Frame it: one class per object, scored per material

- One row per object; the label is its material; the output is one class from a fixed list. The customers' "if you cannot tell, send it to residual" is a confidence requirement a softmax does not meet — say so in the brief rather than hand over the largest probability as if it were one
- The metric is per material: the recall of each, and the purity of each bin, read off a confusion matrix; the baseline is "everything is cardboard"

PITFALL — CLASS IMBALANCE BETWEEN MATERIALS

Cardboard and paper are most of a belt; aluminium is perhaps one object in fifty. Accuracy rewards a network that never names aluminium, and the aluminium bale — the one worth most per tonne — is the one the customer's buyer rejects: a warranty claim. Score every material separately, weight the loss by the inverse frequency or oversample the rare ones, and report bin purity, never one accuracy.

Step 2 — The data: one crop per object, and what a pixel costs

- The camera frame is mostly belt: crop each object out of it, so the network reads the object and not the conveyor
- The weight count of a convolution does not contain the image size, but the activations and the arithmetic do: every feature map is as large as its input, and the dense head grows with the last map
- Choose the side length by looking: the smallest at which an operator can still tell the two plastics apart on the crop — then measure

> **PITFALL — RESOLUTION CHOSEN BY HABIT**
>
> Fed at the camera's full resolution, the early maps are hundreds of times larger than at today's crop size, and one object takes longer than the belt allows. Shrunk to a thumbnail, the label on the bottle and the texture that separates the plastics are gone, the two merge in the confusion matrix, and the plastic bale downgrades. Pick the resolution with the confusion matrix and the clock both open.

Step 3 — Split by plant, then augment the training side only

- Consecutive frames show the same object twice, and a plant’s belt is one distribution: split by plant, never by frame, so the validation plants are ones the network never saw; standardise with the training side’s channel statistics only
- Manufacture views of the training crops: an object lies on the belt at any angle, so rotations and flips keep the label; a shift of position the convolution already handles by equivariance

PITFALL — AUGMENTATION THAT DOES NOT RESPECT THE OBJECT

A hue shift turns green glass into brown, and the label did not survive. A crop that drops the bottle’s neck or the can’s rim removes the feature the operators used. Each transform is a claim that the material is unchanged under it: state the claim, apply the transform to the training loader only, and leave the validation and test crops as the camera took them.

Step 4 — Representation: the crop as a tensor, read by shared weights

- The input is channels × height × width, not a flattened vector: a dense layer producing a map from a flattened crop needs a weight count that contains the square of the image area
- A convolution reads a window with the same weights at every position: a crushed can’s rim is the same evidence wherever it lies on the belt, which is exactly the assumption weight sharing makes — and here it holds
- Pooling then builds invariance: the diverter needs the material, not where on the crop the rim was

> **PITFALL — THE FIRST ATTEMPT FLATTENS**
>
> A multilayer perceptron on flattened crops has to learn the same edge at every position from a few thousand examples, and its first layer does not fit in memory at any resolution that shows the material. Weight sharing is the reason a network fits this belt at all: the same filter at every position, far fewer weights, and the rim learned once.

Step 5 — The model: convolution, batch norm, pooling — and the shapes first

- Blocks of convolution, batch norm and ReLU; a pool after every pair; channels doubling as the map halves; a small dense head with dropout over the materials
- Batch norm after every convolution, because a stack of untreated convolutions on a few thousand crops sits at the loss of a uniform guess and names one material
- Push a dummy batch through the modules one at a time and print every shape before the first training step

> **PITFALL — PADDING LEFT AT ITS DEFAULT**
>
> The default is zero: every map shrinks by the kernel size less one, and the error surfaces as a size mismatch at the dense layer, many layers after the cause, in the middle of a training loop. Set the padding to half the kernel so each map keeps its size, and assert the head’s output shape before anything is trained.

Step 6 — Failure condition: a few thousand crops against millions of parameters

- Nine parameters in ten sit in the dense layer after the flatten, and it can memorise a few thousand crops
- Symptom: the training curve climbs towards the purity the company warrants and the validation curve parts from it and flattens
- Two seeds of the same run differ by points: a comparison smaller than the seed spread is not a comparison

> **PITFALL — OVERFITTING A SMALL SET, READ AS SUCCESS**
>
> The first attempt puts the training accuracy in the release note, and the first customer finds otherwise. Shrink the head — a global pool in place of the flatten — keep the dropout, augment, label more crops from the plants the model gets wrong, and read every number on held-out plants, with the seed spread beside it.

Step 7 — Evaluation: the curve against the clock, the test plants once

- Plot training and validation accuracy against wall-clock seconds, not epochs: the comparison for every new customer's belt is "how good, in how long"
- The learning rate is not a free choice: at the optimiser's default the network plateaus at chance and nothing raises; choose it on the validation plants
- Choose everything on the validation plants; read the test plants once

> **PITFALL — CURVES COMPARED BY EPOCH**
>
> A wider network, or one fed a larger crop, looks better at the same epoch and costs several times the seconds per epoch. The budget is commissioning time, so the x-axis must be time: against the clock the small network has often reached the same point long before the large one, and the comparison reverses.

Step 8 — On the line: latency, release, and the operators' audit

- The crop, the preprocessing and one forward pass must fit inside the fraction of a second before the jet, on the unit's controller — one object at a time, no batch
- Early pooling removes three quarters of every later layer's arithmetic; the convolutions are where the time goes
- Release: new model on one lane, old on the other, bin purity compared on the customer's audit; retrain on the audits as packaging changes; the engineer on call reads each bin's purity against the launch matrix

> **PITFALL — LATENCY MEASURED ON THE DEVELOPMENT MACHINE, IN BATCHES**
>
> Throughput on a graphics card at a batch of sixty-four says nothing about one crop on the unit's controller: a model that clears the batch benchmark misses the jet. Time a single crop end to end on the unit's own hardware, and put that number in every comparison and the specification.

Step 9 — Ship it: the pipeline, and the pitfalls in one table

Camera frame → one crop per object → split by plant → label-preserving augmentation, training only → convolution, batch norm, pooling, small head, shapes asserted → per-material loss, learning rate from validation → curves against the clock → test plants once → one crop timed on the unit → lane against lane, retrain on the audit.

Pitfall	Address it by
Accuracy on an imbalanced belt	recall and bin purity per material; the loss weighted
Resolution chosen by habit	a side length chosen with the confusion matrix open
Augmentation that changes the material	transforms the label survives, on the training loader only
A flattened crop	a convolution: shared weights, invariance from pooling
Padding at its default	padding of half the kernel; the shape check before training
Training accuracy shipped as the result	a global pool, dropout; numbers from held-out plants
Curves compared by epoch	accuracy against wall-clock seconds
Latency measured in batches	one crop end to end on the unit

What we did

1. Counted the weights of a dense layer and a convolutional layer producing the same output, and found the image size absent from the convolutional count
2. Defined translation *equivariance*, the property weight sharing buys, and saw that pooling builds *invariance* on top of it
3. Built a convolutional network — convolution, batch norm, pooling, a dense head — checked its shapes, and found nine parameters in ten in the one dense layer
4. Trained it with a learning rate that is not a free choice, scored the test set once, and quoted the seed spread beside the number
5. Looked inside: the first-layer filters barely moved, and the feature maps lose the photograph as they go down the stack
6. Designed the camera sorter a recycling-robotics company sells for its customers' conveyors in nine steps — per-material recall against an imbalanced belt, a resolution chosen with the clock open, a split by customer plant and augmentation the material survives, shared weights in place of a flattened crop, padding set and shapes asserted, a global pool and dropout against a memorised set, curves read against the clock, and one crop timed on the unit's own controller — each step named for the pitfall it avoids

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 12

five, in the style of the written paper

Exercises — Lecture 12

1 A dense layer mapping a $3 \times 128 \times 128$ input to a $32 \times 128 \times 128$ output has $n_{\text{in}} n_{\text{out}}$ weights. Give the convolutional count for a 7×7 kernel, and say which symbols are absent from it. [5]

2 State the difference between equivariance and invariance, say which of the two pooling builds, and why a species classifier wants it. [4]

3 Convolution is equivariant to translation on the interior of an image but not at the border. Give the reason. [3]

Solutions on Lecture 13’s deck.

Exercises — Lecture 12 (continued)

4 Trained with Adam at its default learning rate of 10^{-3}, the lecture's network neither diverges nor raises an error. State what it does instead, and how the lecture showed it. [4]

5 Two runs of the same network, differing only in the seed, reach 4.22% and 6.76% validation accuracy. A change to the pipeline then gains 1.5 points in a single run. What can you conclude, and what should be reported? [4]

Solutions on Lecture 13's deck.

THE FIVE SET AT THE END OF LECTURE 11

Solutions Lecture 11

not part of this lecture

Solutions — Lecture 11

1. A linear autoencoder $784 \to d \to 784$ with squared loss is trained to convergence on centred data. State…

✓ $\sum_{j>d} \sigma_j^2$, the tail of the spectrum; it reconstructs through the projection onto the span of the leading d right singular vectors — the principal subspace.

- Every reconstruction is a combination of the d rows of the decoder, so it lies in one subspace of dimension at most d, and the closest point of a subspace is the orthogonal projection
- Over all d-dimensional subspaces the projection error is smallest on the principal subspace, where it equals the tail sum; the choice $\mathbf{A} = \mathbf{V}_d, \mathbf{B} = \mathbf{V}_d^\top$ attains it

2. Two linear autoencoders are trained from different seeds to the same loss, and their codes for the same image…

✓ Not a bug. For any invertible $\mathbf{T}$, the encoder $\mathbf{AT}$ with the decoder $\mathbf{T}^{-1}\mathbf{B}$ gives the same reconstruction and a different code: the subspace is unique, its basis is not.

- The loss depends on the product $\mathbf{AB}$ only, and the product does not change under the change of basis
- Gradient descent lands on one basis of the principal subspace; the principal components themselves are one basis among all of them

Solutions — Lecture 11 (2)

3. The code size is set equal to the number of pixels and nothing else constrains the network. State what the…

✓ The identity, with zero loss. The code is a re-encoding of the pixels that loses nothing and learns nothing.

- With $d = n$ the tail of the spectrum is empty, so the floor is zero and the identity reaches it
- A bottleneck narrower than the input is the only thing that forbids copying; without it the code carries exactly the pixels, differently arranged

4. An anomaly detector scores images by their reconstruction error from an autoencoder trained on nine classes;…

✓ Reconstruction error measures how well the training classes can imitate an image, not how rare it is. A held-out class the training classes resemble is reconstructed as well as they are, so its errors sit on top of theirs. Examples of the anomalies, to measure the detector on.

- A coat held out from pullovers, shirts and T-shirts decodes as a dark pullover; nothing in the score knows it was never seen
- Whether the method works is a property of what the anomalies look

Solutions — Lecture 11 (3)

5. Nearest-neighbour retrieval in a 32-number autoencoder code scores the same precision at ten as in the 784…

✓ A twenty-fourfold saving in storage and in the cost of every comparison, at equal precision. It could not beat the pixels because the reconstruction loss contains no information about which images are similar: the code is a compressed account of pixel distance.

- A nearest-neighbour search compares the query with every item in the catalogue, so the length of the code is the cost of the search
- A representation that is better than its input at a task needs a training signal about that task, and reconstructing the input supplies none

LECTURE 13 · GÉRON CH. 12

Transfer learning

Why features learned on a large labelled corpus transfer to these flowers
Freeze the backbone and train a head; then let the last block move, at its own learning rate
Augmentation — and why it belongs to the training loader only

Run the Lecture 12 notebook first